Top-Down Parsing

1

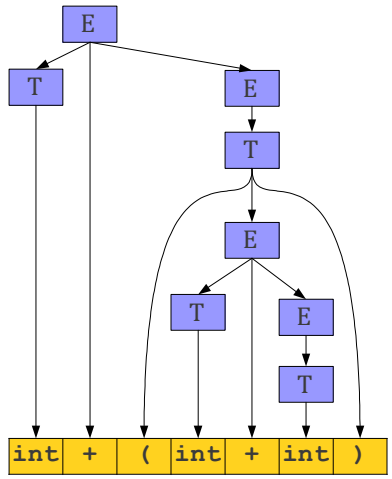
Different Types of Parsing

- **Top-Down Parsing**
  - Beginning with the start symbol, try to guess the productions to apply to end up at the user's program.
- **Bottom-Up Parsing**
  - Beginning with the user's program, try to apply productions in reverse to convert the program back into the start symbol.

2

Top-Down Parsing

E → T
E → T + E
T → int
T → (E)



3

Challenges in Top-Down Parsing

- Top-down parsing begins with virtually no information.
  - Begins with just the start symbol, which matches *every* program.
- How can we know which productions to apply?
- In general, we can't.
  - There are some grammars for which the best we can do is guess and backtrack if we're wrong.
  - If we have to guess, how do we do it?

4

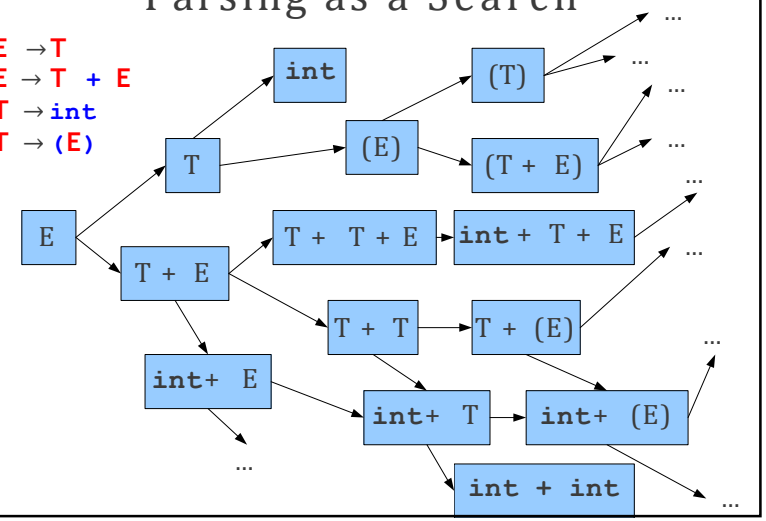
Parsing as a Search

- An idea: **treat parsing as a graph search**.
- Each node is a **sentential form** (a string of terminals and nonterminals derivable from the start symbol).
- There is an edge from node α to node β iff $\alpha \Rightarrow \beta$.

5

Parsing as a Search

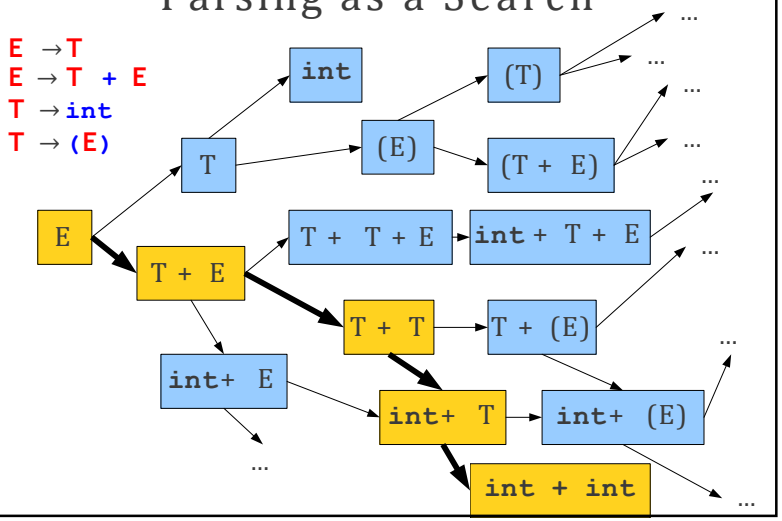
E → T
E → T + E
T → int
T → (E)



6

Parsing as a Search

E → T
E → T + E
T → int
T → (E)



7

Our First Top-Down Algorithm

- **Breadth-First Search**
- Maintain a worklist of sentential forms, initially just the start symbol **S**.
- While the worklist isn't empty:
  - Remove an element from the worklist.
  - If it matches the target string, you're done.
  - Otherwise, for each possible string that can be derived in one step, add that string to the worklist.
- Can recover a parse tree by tracking what productions we applied at each step.

8

Leftmost Derivations

- Recall: A **leftmost derivation** is one where we always expand the leftmost symbol first.
- Updated algorithm:
  - Do a breadth-first search, **only considering leftmost derivations**.
    - Dramatically drops branching factor.
    - Increases likelihood that we get a prefix of nonterminals.
  - Prune sentential forms that can't possibly match.
    - Avoids wasted effort.

9

Leftmost DFS

- Idea: Use **depth-first** search.
- Advantages:
  - Lower memory usage: Only considers one branch at a time.
  - High performance: On many grammars, runs very quickly.
  - Easy to implement: Can be written as a set of mutually recursive functions.

10

Left Recursion

- A nonterminal **A** is said to be **left-recursive** iff

$$A \Rightarrow^* A\omega$$

  for some string ω.
- Leftmost DFS may fail on left-recursive grammars.
- Fortunately, in many cases it is possible to eliminate left recursion (see Handout 08 for details).

11

Summary of Leftmost BFS/DFS

Leftmost BFS	Leftmost DFS
Leftmost BFS works on all grammars.	Leftmost DFS works on grammars without left recursion.
Worst-case runtime is exponential.	Worst-case runtime is exponential.
Worst-case memory usage is exponential.	Worst-case memory usage is linear.
Rarely used in practice.	Often used in a limited form as **recursive descent**.

12

Predictive Parsing

13

Predictive Parsing

- The leftmost DFS/BFS algorithms are **backtracking** algorithms.
  - Guess which production to use, then back up if it doesn't work.
  - Try to match a prefix by sheer dumb luck.
- There is another class of parsing algorithms called **predictive** algorithms.
  - Based on remaining input, predict (*without backtracking*) which production to use.

14

Tradeoffs in Prediction

- Predictive parsers are *fast*.
  - Many predictive algorithms can be made to run in linear time.
  - Often can be table-driven for extra performance.
- Predictive parsers are *weak*.
  - Not all grammars can be accepted by predictive parsers.
- Trade *expressiveness* for *speed*.

15

Exploiting Lookahead

- Given just the start symbol, how do you know which productions to use to get to the input program?
- Idea: Use **lookahead tokens**.
- When trying to decide which production to use, look at some number of tokens of the input to help make the decision.

16

Implementing Predictive Parsing

- Predictive parsing is only possible if we can predict which production to use given some number of lookahead tokens.
- Increasing the number of lookahead tokens increases the number of grammars we can parse, but complicates the parser.
- Decreasing the number of lookahead tokens decreases the number of grammars we can parse, but simplifies the parser.

17

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

18

Predictive Parsing

E

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

19

Predictive Parsing

E

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

20

Predictive Parsing

E
T + E

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

21

Predictive Parsing

E
T + E

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

22

Predictive Parsing

E
T + E

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

23

Predictive Parsing

E
T + E
int + E

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

24

Predictive Parsing

E
T + E
int + E

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

25

Predictive Parsing

E
T + E
int + E

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

26

Predictive Parsing

E
T + E
int + E

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

27

Predictive Parsing

E
T + E
int + E
int + T

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

28

Predictive Parsing

E
T + E
int + E
int + T

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

29

Predictive Parsing

E
T + E
int + E
int + T

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

30

Predictive Parsing

E
T + E
int + E
int + T
int + (E)

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

31

Predictive Parsing

E
T + E
int + E
int + T
int + (E)

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

32

Predictive Parsing

E
T + E
int + E
int + T
int + (E)

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

33

Predictive Parsing

E
T + E
int + E
int + T
int + (E)

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

34

Predictive Parsing

E
T + E
int + E
int + T
int + (E)
int + (T + E)

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

35

Predictive Parsing

E
T + E
int + E
int + T
int + (E)
int + (T + E)

E → T
E → T + E
T → int
T → (E)

int	+	(	int	+	int	)

36

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)

int	+	(	int	+	int	)

37

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)

int	+	(	int	+	int	)

38

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)

int	+	(	int	+	int	)

39

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)

int	+	(	int	+	int	)

40

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)

int	+	(	int	+	int	)

41

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)

int	+	(	int	+	int	)

42

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)

int	+	(	int	+	int	)

43

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)

int	+	(	int	+	int	)

44

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)
int + (int + int)

int	+	(	int	+	int	)

45

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)
int + (int + int)

int	+	(	int	+	int	)

46

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)
int + (int + int)

int	+	(	int	+	int	)

47

Predictive Parsing

E → T
E → T + E
T → int
T → (E)

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)
int + (int + int)

int	+	(	int	+	int	)

48